Some examples that we mentioned in section 3.2 are 2-3 trees[6] (shown in figure 3.12) and red-black trees[7] (figure 3.13).

The algorithms to maintain the constraints for these trees, however, tend to be quite complicated, so much so that, for instance, many textbooks on algorithms omit the delete method altogether.

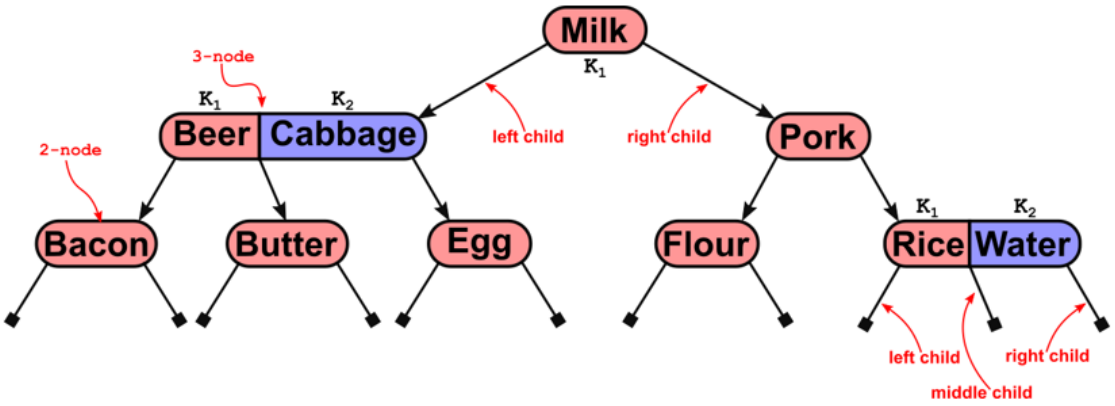


Figure 3.12 A 2-3 tree containing the keys we used in our grocery store example. Nodes in 2-3 trees can contain one or two keys, sorted ascendingly, and respectively 2 or 3 links; besides left and right children, 3-nodes also have a middle child: all keys K in the subtree pointed to by a middle link must obey this condition: $K_1 > K >= K_2$, where K_1 and K_2 are the first and second key, respectively, of the 3-node.
2-3 trees are guaranteed to be balanced by the way insertion is performed: keys are added to the leaves, and when a leaf grows to 3 elements, it's split and the middle element is bubbled up to the parent node (that's also possibly recursively split). It is guaranteed that the height of a 2-3 tree with n keys is between $\log_2(n)$ and $\log_3(n)$.

Turns out, quite surprisingly, that we can use treaps, which seems quite unbalanced, to obtain tendentially-balanced[8] binary search